

Using LLMs to Translate and Localize *Language Landscapes*

Draft for Koike. The goal is not to learn LaTeX first. The goal is to give an LLM the book source, make it work inside a controlled folder, and use it as a careful translation assistant, LaTeX assistant, terminology checker, and reviewer. ## Short Version Use a coding-agent app rather than an ordinary chat window if possible. The ordinary chat versions of Claude, ChatGPT, Gemini, etc. can help with phrasing, but they are weaker for this project because they cannot reliably manage a folder of .tex files, preserve LaTeX commands, compare versions, and check the final output.

The best default is to use **both** Codex and Claude Code. They often catch different problems, and the cross-review is valuable:

- **Codex app:** good for working directly in a project folder, reviewing diffs, creating helper files such as a Japanese house style, and asking the model to check the LaTeX source.
- **Claude Code / Claude Code app:** good for the same kind of project-folder work, especially if using Claude's strongest coding model.

If you are comfortable with the terminal, the CLI versions are cleanest. If not, use the apps. The important thing is that the model has access to a copy of the folder containing the LaTeX source, not only to the PDF. ## What Brett Has Sent Brett has already sent the PDF of the book and the Chapter 18 sample PDF. He will also send the LaTeX source:

- the PDF of the book, for reading and orientation;
- the Chapter 18 sample PDF, as a model of what an LLM-localized standalone chapter can look like;
- a zipped LaTeX folder, which contains the source files that should be translated.

For LLM-assisted translation, the .tex files are more important than the PDF. The PDF shows the finished book, but the .tex source is what the model can edit while preserving examples, references, macros, and formatting.

If Koike wants to build PDFs from the .tex source, he can ask Codex or Claude to check whether the needed LaTeX tools are installed and to install or configure whatever is missing, with his approval. He does not need to understand the LaTeX build system before beginning, but he should approve installations and permissions deliberately. If that part is uncomfortable, it is worth doing once with help from Brett or another technically comfortable person. ## Folder Setup Create a new project folder, for example:

Language_Landscapes_Japanese/

Unzip the LaTeX source into that folder. Do not work in the original zip file. Keep an untouched backup copy of the zip somewhere else.

Recommended folder contents:

```
Language_Landscapes_Japanese/  
  chapters/  
  figures/  
  localcommands.tex  
  localmetadata.tex  
  localpackages.tex  
  main.tex  
  AGENTS.md  
  japanese-house-style.md  
  translation-notes.md
```

The files AGENTS.md, japanese-house-style.md, and translation-notes.md can be created with the LLM. They are there to help the LLM remember the translation rules. ## Reviewing Markdown Files Much of the project guidance will live in Markdown files ending in .md. These are plain-text files, but they are easier to read in a Markdown viewer than in a raw text editor.

If possible, ask Codex or Claude to install **Roughdraft** and set it up so Markdown files are easy to review. Then use it for files such as:

- AGENTS.md;
- japanese-house-style.md;
- translation-notes.md;
- terminology plans;
- reviewer reports.

Roughdraft is useful because it lets Brett or Koike comment directly on a Markdown file, mark passages, suggest edits, and then give the file back to the LLM as reviewed guidance. This is especially helpful for terminology and house-style decisions, where a small wording choice can affect many chapters.

Once Koike has either Codex or Claude Code working, he can ask the model to do most of the remaining setup: create the folder structure, create the guidance files, install or configure Roughdraft if appropriate, and set up a Markdown viewer or browser preview. The goal is simple: when the LLM creates a Markdown plan or terminology table, Koike should be able to open it, read it comfortably, comment on it, and give the file back to the LLM as reviewed guidance.

If Roughdraft is not available, set up a simpler fallback:

- choose a Markdown editor or viewer, such as VS Code, Obsidian, Typora, Marked, or another familiar app;
- configure .md files to open in that app by default;
- if using a browser, install or enable a Markdown-preview extension so .md files render as formatted documents rather than raw text;
- keep all reviewed Markdown files inside the project folder so Codex or Claude can read the comments and update the translation guidance.

For a non-terminal workflow, this small setup step matters. The translation will produce many reviewable planning files, and Koike should not have to read raw Markdown syntax if that gets in the way. ## Permissions When the app asks for folder access, give it access only to the project folder you created for this translation. It does not need access to your whole computer.

Approve normal actions such as:

- reading .tex, .md, .bib, and figure files in the project folder;
- creating or editing translated .tex files;
- creating notes, style guides, and terminology tables;
- running a LaTeX build if the required tools are installed.

Be more cautious about:

- deleting files;
- changing files outside the project folder;
- installing new software;
- uploading the full source folder to third-party services without permission;
- running commands you do not understand.

If you are using Codex, the app is designed around a project folder, approvals, and sandboxing. Its current guidance recommends starting with default permissions and loosening them only for trusted folders. Claude Code has a similar permission system: approve file and command access deliberately, especially at the beginning. ## Recommended First Prompt Start with a planning prompt, not a translation prompt:

I am translating the book Language Landscapes from English into Japanese.
This is a localization, not a literal translation.

Please inspect the project folder and identify:

1. the chapter files;
2. the LaTeX macros and environments that must not be changed;
3. the files that should be translated first;
4. the files that should be left alone;
5. a safe workflow for translating one chapter at a time.

Do not edit files yet. First produce a plan and a list of questions.

After the plan looks sensible, ask the model to create the project guidance files:

Create AGENTS.md, japanese-house-style.md, and translation-notes.md for this project.

They should instruct LLMs to:

- translate prose into natural Japanese;
- preserve LaTeX command names and syntax;
- preserve English object-language examples where English is being analyzed;
- translate numbered-example labels, interlinear glosses, and natural translations for Japanese readers;
- keep technical terms consistent;
- give the English term in parentheses at first occurrence of important terms;
- add brief warnings or analogies where a Japanese term might mislead readers.

Do not translate a chapter yet.

Core Translation Policy

This book is about English grammar. That means many English words and sentences in examples are not prose to be translated. They are object-language material being analyzed.

For example, if the book discusses the English word *that*, the Japanese translation should normally keep that as the object-language form. The surrounding explanation should be in Japanese.

Use this distinction:

- **Prose:** translate into natural Japanese.
- **English examples under analysis:** usually keep in English.
- **Technical terms:** translate into Japanese, with the English term in parentheses at first occurrence.
- **LaTeX commands:** do not translate command names such as `\chapter`, `\section`, `\term`, `\mention`, `\ea`, `\ex`, `\gll`, `\glt`, or `\z`.
- **Citations and labels:** do not casually change citation keys, labels, or cross-reference commands.

Localization, Not Literal Translation

The translation should not sound like English with Japanese words inserted. It should read as a Japanese educational text for readers who are learning about English grammar.

But it also should not erase the author's analysis. The book makes specific distinctions, and those distinctions must survive in Japanese:

- category vs function;
- clause vs phrase;
- head-dependent structure vs non-headed coordination;
- main clause vs matrix clause;
- content clause vs relative clause vs comparative clause;

- subordinator/coordinator as categories vs marker as a function;
- ellipsis vs gap;
- sentence-initial discourse connector vs syntactic coordination.

Some Japanese terms are familiar but risky. When a term may mislead readers, add a brief warning, analogy, or diagnostic. This extra material is allowed. The aim is not to reproduce every English sentence mechanically, but to produce the Japanese edition that teaches the same analysis clearly.

Example:

Translate “coordination” consistently as 等位接続, but do not let readers understand it as mere 並列 or loose li

Japanese House Style

Create a file called `japanese-house-style.md`. It should record decisions like these:

English	Japanese default	Note
clause	節	Keep distinct from sentence and phrase.
phrase	句	Do not blur with 文 or 表現.
main clause	主節	Not the same as matrix clause.
matrix clause	母節	The clause that contains another clause.
subordinate clause	従属節	Relational term.
subordination	従属化 / 従属関係	Use according to whether process or relation is meant.
coordination	等位接続	Avoid broad 並列 as the main term.
coordinator	等位接続詞	Category term.
subordinator	従属接続詞	Category term.
marker	標識	Function term.
complement	補部	Prefer over school-grammar 補語 for this book.
modifier	修飾語	Keep consistent.
adjunct	付加部	Keep distinct from complement.
content clause	内容節	Do not define as “complement clause.”
comparative clause	比較節	Watch ellipsis examples.
relative clause	関係節	Mainly treated in the next chapter.
ellipsis	省略	Distinguish from gap.
gap	空所	A structural position, not just omitted recoverable meaning.
mandative construction	mandative 構文 (命令・要求構文)	Keep English term visible.
present subjunctive	仮定法現在	Do not suggest counterfactual meaning.

The house style should also specify:

- use polite Japanese if that is the chosen voice;
- avoid stiff translationese;
- keep English grammatical examples in English unless they are merely illustrative and not being analyzed;
- use Japanese natural translations in numbered examples;
- avoid italics for Japanese text unless the LaTeX style requires it;

- record every terminology decision as it is made.

Possible Chapter Workflow

This is one possible workflow, not a rule. Koike should adapt it to his own habits and to the kind of Japanese edition he wants. The main caution is practical: asking the model to translate the whole book in one pass will make terminology and examples harder to control. Chapter-by-chapter work, and sometimes section-by-section work, is usually easier to review.

One useful sequence for each chapter is:

1. **Terminology pass:** ask for a list of key terms and proposed Japanese equivalents.
2. **Risk pass:** ask which terms or examples are likely to mislead Japanese readers.
3. **Draft translation:** translate one section while preserving LaTeX.
4. **LaTeX check:** ask the model to check for broken commands, braces, labels, and environments.
5. **Terminology check:** ask it to compare the chapter against `japanese-house-style.md`.
6. **Example check:** ask it to inspect every numbered example.
7. **Review-board pass:** ask several reviewer roles to critique the chapter.
8. **Revision:** apply only the comments that genuinely improve the translation.
9. **Build or visual check:** if possible, compile the PDF or ask a LaTeX-capable assistant to do so.

Numbered Examples

Numbered examples need special care. The model should not simply translate the first visible line.

For interlinear examples:

- the top line usually stays in English if English is the object language;
- the gloss line should be useful to Japanese readers, not a leftover English gloss;
- grammatical tags such as PST, PRS, PL, DEF, NEG, AUX, Sdr, Crd, and Mk can remain as tags if the edition uses them consistently;
- the natural translation line should be Japanese.

Ask the model to check every example with a prompt like this:

Audit all numbered examples in this chapter.

For each example, check:

1. whether the English object-language line was preserved where it should be;
2. whether the interlinear gloss line is localized for Japanese readers;
3. whether the natural translation is in Japanese;
4. whether the number of tokens in the example line and gloss line still aligns;
5. whether brackets, stars, labels, and ellipses were preserved;
6. whether any English function word accidentally remains in the gloss line.

Report problems before editing.

Model Strategy

Use the strongest models for the high-stakes stages:

- terminology planning;

- first full draft of a difficult section;
- review of examples and glosses;
- final critique before sharing the chapter.

For Codex, use the current recommended top model available in Codex and choose the highest reasoning effort for difficult translation and review tasks. Current Codex documentation recommends gpt-5.5 for complex coding, knowledge work, and research workflows, and Extra High effort for long, reasoning-heavy tasks.

For Claude Code, use the strongest Opus model available to you for hard translation/review work, with xhigh effort where available. Official Claude documentation currently identifies Opus 4.8 as the strongest Opus-tier model and says coding and high-autonomy work should set xhigh explicitly. If Opus is unavailable or too expensive, use the strongest Sonnet model available and reserve Opus for the final review.

Other models such as Kimi, Qwen, or GLM can be useful as minority reviewers. Do not let them become the source of truth. Use them to find awkward Japanese, possible terminological alternatives, or places where the explanation may confuse readers. Then ask Codex or Claude to evaluate those comments against the house style and the English source. ## Cross-Review Workflow A strong pattern is to make different models disagree productively.

Example sequence:

1. Claude produces a first Japanese draft of one section.
2. Codex reviews the draft against the English .tex, the house style, and the LaTeX constraints.
3. Claude revises using Codex's critique.
4. Codex checks only the changed passages and numbered examples.
5. A third model gives a reader-response critique: "What would confuse a Japanese teacher trainee?"
6. Codex or Claude decides which third-model comments are valid and revises selectively.

Use prompts like:

Review this Japanese translation against the English source.

Focus on:

- mistranslations;
- loss of technical distinctions;
- unnatural Japanese;
- places where a Japanese reader may import the wrong school-grammar concept;
- LaTeX damage.

Do not rewrite the whole chapter. Give prioritized findings with source references.

And:

Now act as a Japanese grammar-education reviewer.

Assume the reader is an English teacher or teacher trainee in Japan.

Where might the translation cause misconstrual or misapplication?

Recommend brief additions only where they are pedagogically necessary.

Review Board Roles

Ask for several focused reviews rather than one vague review.

Useful roles:

- **Terminology reviewer:** checks consistency and first-occurrence English terms.
- **Japanese style reviewer:** checks naturalness, sentence rhythm, and translationese.
- **English grammar reviewer:** checks whether the analysis survived.

- **Examples/glosses reviewer:** checks numbered examples, interlinear glosses, natural translations, stars, brackets, and ellipses.
- **LaTeX reviewer:** checks syntax, macros, labels, cross-references, and command damage.
- **Pedagogical reviewer:** checks whether the chapter will help Japanese readers rather than merely sound correct.

Ask each reviewer to give findings, not praise. The best review starts with problems. ## What Not to Do Avoid these patterns:

- translating from the PDF only;
- translating the whole book in a single prompt;
- letting the model translate LaTeX command names;
- accepting a fluent translation before checking technical terms;
- letting English gloss words remain in interlinear gloss lines by accident;
- removing brackets, stars, labels, or ellipses from examples;
- flattening `matrix` clause and `main` clause into one Japanese term;
- using familiar Japanese grammar terms without checking whether they match this book’s analysis;
- accepting all suggestions from every model;
- assuming that a model’s confident terminology is correct.

A Good Chapter Prompt

Translate Chapter 18 into Japanese.

Before translating, read:

- AGENTS.md
- `japanese-house-style`.md
- translation-notes.md
- the English chapter source

Requirements:

- This is localization, not literal translation.
- Preserve LaTeX command names, labels, citations, examples, and structure.
- Translate prose into natural Japanese.
- Preserve English object-language examples when English forms are being analyzed.
- Translate example labels and natural translations into Japanese.
- For interlinear examples, localize lexical glosses for Japanese readers while preserving grammatical tags.
- At first occurrence of important technical terms, use Japanese plus the English term in parentheses.
- Add brief warnings, analogies, or diagnostics only where Japanese terminology is likely to mislead.
- Do not silently collapse technical distinctions such as `main/matrix`, `category/function`, `clause/phrase`, and `e`

Work section by section. After each section, report any terminology decisions and any LaTeX risks.

Final Quality Checklist

Before a translated chapter is accepted, ask the model to confirm:

- all technical terms match `japanese-house-style`.md;
- first occurrences include the English term where required;
- English object-language forms remain English;

- Japanese prose is natural and not just English syntax in Japanese words;
- all numbered examples are intact;
- all interlinear glosses have token alignment;
- all natural translations are Japanese;
- no accidental English function words remain in localized gloss lines;
- brackets, stars, ellipses, and example labels are preserved;
- no LaTeX commands, labels, citations, or environments were damaged;
- any added localization material is brief and justified.

Current Reference Notes

Checked on 2026-06-23:

- Codex documentation: Codex app supports project folders, diff review, permissions/sandboxing, AGENTS.md, model selection, and Extra High reasoning for difficult tasks.
- Claude documentation: Claude Code supports permission controls; Anthropic's current model docs identify Opus 4.8 as the strongest Opus-tier model and recommend explicit `xhigh` effort for coding and high-autonomy work.